

VirtuEx 성능 개선 감사 보고서 (5차)

VirtuEx Performance Improvement Audit Report (5th)

- 감사일: 2026-03-16
- 감사 범위: 주문 엔진 매칭 알고리즘, 프론트엔드 데이터 페칭/폴링, WebSocket 재접속, DB 테이블 관리, 캔들스틱 집계, 환율 캐싱
- 감사 방법: 소스 코드 정적 분석, 알고리즘 복잡도 분석, 쿼리 패턴 추적, 번들/캐싱 전략 검토
- 심각도 기준: Critical(즉시 수정) / High(1주 내) / Medium(2주 내) / Low(개선 권장)
- 비고: 발견 사항 정리만 수행하며, 수정은 포함하지 않음

이전 감사 대비 수정 현황 (4차 → 5차)

이전 이슈	상태	비고
[FR-H-01] AssetList useMemo 의존성 과다	미수정	추후 개선 예정
[FR-H-02] 리더보드 FLIP 애니메이션 리플로우	미수정	추후 개선 예정

요약 (Executive Summary)

카테고리	Critical	High	Medium	Low	합계
A. 주문 엔진 성능	0	1	1	0	2
B. 프론트엔드 폴링/캐싱	0	0	2	1	3
C. 캔들스틱 집계	0	0	1	0	1
D. DB 테이블 관리	0	1	1	0	2
E. 환율/공유 캐시	0	0	0	1	1
F. WebSocket 재접속	0	0	0	1	1
합계	0	2	5	3	10

A. 주문 엔진 성능 — 2건

[OE-H-01] 오더북 삽입 시 전체 정렬 — $O(N \log N)$ 비효율 (High)

현상: `insertEntry()` 함수에서 주문 삽입 후 `Array.sort()` 를 매번 호출. N개 주문이 있는 오더북에서 삽입당 $O(N \log N)$ 연산이 발생하며, 이진 탐색 삽입($O(\log N)$) 대비 비효율적 위치: `backend/services/order-engine/src/domain/services/matching-engine.service.ts:124-143` 영향: 대량 주문 유입 시 매칭 엔진 지연 가능 권장: 이진 탐색(`bisect`)으로 삽입 위치를 찾아 `splice()` 로 $O(N)$ 삽입, 또는 정렬 트리/스킵 리스트 자료구조로 $O(\log N)$ 달성 심각도: High

[OE-M-01] 주문 체결 시 `toRemove.includes()` — $O(N*M)$ 비효율 (Medium)

현상: `entries.filter((e) => !toRemove.includes(e.orderId))` 에서 `includes()` 는 $O(M)$ 탐색이므로 전체 복잡도가 $O(N*M)$. 대량 체결 시 성능 저하 위치: `backend/services/order-`

engine/src/domain/services/matching-engine.service.ts:229-233 권장: `const removeSet = new Set(toRemove)` 로 변환 후 `removeSet.has()` 로 O(1) 탐색 심각도: Medium

B. 프론트엔드 폴링/캐싱 — 3건

[FC-M-01] 포트폴리오 혹은 `staleTime` 미설정으로 중복 요청 (Medium)

현상: `usePortfolio()` 와 `usePortfolioValuation()` 모두 10초 폴링 (`refetchInterval: 10000`)이지만 `staleTime` 미설정. 글로벌 `staleTime: 5000` 에 의해 컴포넌트 마운트마다 즉시 재요청 발생. 두 혹은 유사 데이터를 별도 요청 위치: frontend/src/hooks/usePortfolio.ts:102-113, 122-132 권장: `staleTime: 8000` 설정으로 폴링 주기 내 중복 요청 방지. 두 혹은 통합 검토 심각도: Medium

[FC-M-02] 주문 내역 혹은 `WebSocket` 이벤트 무효화와 10초 폴링 중복 (Medium)

현상: `useOrders()` 와 `useTradeHistory()` 모두 10초 폴링 + `WebSocket` 이벤트 무효화 사용. `WebSocket`으로 실시간 업데이트를 이미 받고 있어 10초 폴링은 과도 위치: frontend/src/hooks/useOrders.ts:42-74, 179-201 권장: `staleTime: 8000` 추가, 폴링 주기를 30초로 완화 (`WebSocket`이 즉시 무효화 처리) 심각도: Medium

[FC-L-01] 시세 데이터 동시 요청 중복 가능성 (Low)

현상: `useMarketPrices()` 는 `staleTime: 290000` 으로 적절히 설정되었으나, 대시보드+마켓티커+스포츠타이트검색이 동시 마운트 시 TanStack Query 디듀플리케이션 전에 병렬 요청 가능 위치: frontend/src/hooks/useMarket.ts:22-36 권장: React Query DevTools로 실제 중복 요청 여부 확인. 필요 시 `queryClient` 단위 디듀플리케이션 검증 심각도: Low

C. 캔들스틱 집계 — 1건

[CS-M-01] 대형 인터벌 캔들스틱 — 클라이언트 사이드 집계 (Medium)

현상: 4시간/1일 인터벌의 경우 클라이언트가 최대 5000개 1분봉을 가져와 JavaScript로 집계. 모바일 기기에서 상당한 연산 부하. `fetchLimit` 이 `4h= limit*240` , `1d= limit*1440` (최대 5000/2000) 위치: frontend/src/hooks/useMarket.ts:149-219 권장: 백엔드 서버에서 상위 인터벌 캔들을 사전 계산/캐싱하여 제공 (Candlestick 모델 활용) 심각도: Medium

D. DB 테이블 관리 — 2건

[DB-H-01] PriceHistory 테이블 — 파티셔닝/보관 정책 미구현 (High)

현상: 스키마에 TODO 주석으로 "타임스탬프 기반 파티셔닝 및 90일 데이터 보관 정책 적용" 명시되어 있으나 미구현. PriceHistory는 쓰기 집중 테이블로 무한 증가 위치: backend/services/market-data/prisma/schema.prisma:30-31 권장: PostgreSQL 시간 기반 테이블 파티셔닝 구현 + 크론 잡으로 90일 초과 데이터 삭제 심각도: High

[DB-M-01] PageView 테이블 — 보관 정책 미구현 (Medium)

현상: TODO 주석에 "30일 데이터 보관 정책(크론 잡)" 명시되었으나 미구현. 모든 페이지 뷰를 기록하며 정리 없이 테이블 무한 증가 위치: backend/services/user-auth/prisma/schema.prisma:222-233 권장: 30일 초과 레코드 삭제하는 스케줄 작업 구현 심각도: Medium

E. 환율/공유 캐시 — 1건

[EC-L-01] 환율 조회 — 서비스별 독립 캐싱으로 중복 API 호출 (Low)

현상: order-engine과 portfolio 서비스가 각각 독립적으로 USD-KRW 환율을 외부 API에서 가져와 10분 TTL로 캐싱. 동일 데이터에 대한 외부 API 호출 중복 **위치:** order-engine/src/application/services/order.service.ts:51 , portfolio/src/domain/services/balance.service.ts:71 **권장:** Redis에 환율 데이터를 중앙 캐싱하여 모든 서비스가 공유 **심각도:** Low

F. WebSocket 재접속 — 1건

[WS-L-01] 재접속 시 지수 백오프 미적용 — 썬더링 허드 위험 (Low)

현상: Socket.IO 재접속이 reconnectionDelay: 1000 , reconnectionAttempts: 10 으로 고정. reconnectionDelayMax 미설정으로 서버 과부하 시 모든 클라이언트가 동일 주기로 재접속 시도 (썬더링 허드) **위치:** frontend/src/hooks/useWebSocket.ts:107-114 **권장:** reconnectionDelayMax: 10000 및 randomizationFactor: 0.5 설정으로 지터 추가 **심각도:** Low

종합 의견

5차 성능 감사에서는 총 10건의 이슈가 발견되었습니다. 4차 감사에서 발견된 AssetList useMemo 의존성 및 리더보드 FLIP 애니메이션 이슈는 아직 미수정 상태입니다.

High 2건 중 주문 엔진의 오더북 삽입 알고리즘($O(N \log N)$)은 대량 주문 처리 시 병목이 될 수 있으며, PriceHistory 테이블 파티셔닝 미구현은 장기 운영 시 DB 성능에 직접적 영향을 미칩니다.

프론트엔드 폴링 최적화(staleTime 설정)와 캐들스틱 서버사이드 집계 이전은 사용자 체감 성능 및 모바일 환경 개선에 기여할 것입니다.

본 보고서는 자동 생성된 감사 결과이며, 수정 사항은 포함되지 않습니다.